

45-Day AI Engineering Study Plan

Detailed Day-by-Day Reference Guide

Ed Donners + AI Engineering from Scratch + AI-Engineering Roadmap
Interleaved 1.5-month full-time curriculum, 8.5 hours/day

Resource	Your Time	How
Ed Donners (33h video + 8 projects)	~5 days	1.5x playback, follow along, ship projects
AI-Eng Roadmap (12 projects)	~22 days	AI generates boilerplate, you learn architecture + verify
From-scratch [CORE] lessons only	~6 days	Read + code key lessons, skip [DEEP]/[SKIP]
Buffer / re-dos / interview prep	~12 days	Things will go wrong; 2mo version has 15 extra days
TOTAL	~45 days	= 1.5 months at 8.5h/day

Each day shows: exact Ed Donners lectures to watch, from-scratch lessons to read/code, roadmap project phases to build, time allocation, and end-of-day deliverable.

Contents

1. How to Read This Document
2. Legend and Conventions
3. Phase 1: Foundations (Days 1-5)
4. Phase 2: RAG Deep Dive (Days 6-15)
5. Phase 3: Agents (Days 16-24)
6. Phase 4: Evaluation + Infrastructure (Days 25-34)
7. Phase 5: Capstone (Days 35-45)
8. 2-Month Extension (Days 46-60)
9. AI Code Generation Workflow
10. What to Ruthlessly Skip

1. How to Read This Document

This document gives you a granular, hour-by-hour plan for each of the 45 days. Every day is broken into three time blocks matching the interleaving strategy:

Morning (3h): Ed Donners video lectures at 1.5x playback speed. Follow along in your editor. Build the project he builds.

Afternoon (3h): AI-Engineering Roadmap project. Read the IMPLEMENTATION_PLAN.md, use AI to generate code, verify you understand every line.

Evening (2.5h): From-scratch [CORE] lessons. Read the lesson, code the key examples, understand the math behind what you built in the morning.

Some days deviate from this pattern (e.g., when Ed Donners has no corresponding content, the full day goes to roadmap projects). Each day explicitly states the time allocation.

2. Legend and Conventions

W# = Ed Donners Udemmy course week number

P#-L## = AI Engineering from Scratch, Phase #, Lesson ## (with full title)

R-P# = AI-Engineering Roadmap Project # (with specific phase from IMPLEMENTATION_PLAN.md)

[CORE] = study alongside | **[DEEP]** = optional depth | **[SKIP]** = you know this

Time blocks are labeled by resource:

Ed Donners = video lectures | **From-Scratch** = theory lessons | **Roadmap** = project building | **Buffer** = review/polish

Each day ends with a **Deliverable** - what you must have produced by end of day.

3. Phase 1: Foundations (Days 1-5)

Ed Donners Weeks 1-2. Core concepts: API calls, tokenization, transformers, tool calling, multimodal. First roadmap project (P1: Advanced RAG) setup begins on Day 5.

Day 1

Time	Activity	Detail
3h	Ed Donners	W1 Lectures 1-10: Course intro, Ollama setup, first local LLM call, OpenAI API setup, chat completions, system vs user prompts, your first LLM product. Follow along: install Ollama, pull a model, make API calls. Watch at 1.5x. Take notes on: token costs, context windows, model selection criteria.
3h	Ed Donners	W1 Lectures 11-20: Website summarizer project, frontier models compared (GPT-4o, Claude, Gemini, Grok), transformers architecture overview (high-level), tokens and tokenization with tiktoken, context windows explained. Build the website summarizer alongside Ed.
2.5h	Ed Donners	W1 Lectures 21-37: Chaining GPT calls together, brochure generator (Project 1), environment setup, Cursor/UV/Git walkthrough [SKIP if you know this], first look at reasoning models. Finish the brochure generator. This is your first shipped LLM product.

Deliverable: Brochure generator project complete. Ollama + OpenAI API working. Notes on token costs and model selection.

Day 2

Time	Activity	Detail
3h	From-Scratch	P7-L02: Self-Attention from Scratch [CORE]. Build the attention mechanism by hand in Python. Code: Q, K, V matrices, softmax($QK^T/\sqrt{d}$))V. Understand WHY attention works before Ed explains it conceptually. Read the lesson, code every example, run it. This is the single most important from-scratch lesson.
3h	From-Scratch	P7-L03: Multi-Head Attention [CORE]. Extend single-head to multi-head. Code: split into h heads, parallel attention, concatenate, linear projection. Understand why multi-head captures different representation subspaces.
2.5h	From-Scratch	P7-L05: The Full Transformer: Encoder + Decoder [CORE]. See how attention fits into the full architecture. Code: positional encoding, encoder block, decoder block, masked attention. By end of session you should be able to draw the transformer architecture from memory.

Deliverable: Self-attention + multi-head attention code working. Full transformer architecture understood. Can explain Q/K/V to someone without looking at code.

Day 3

Time	Activity	Detail
3h	Ed Donners	W2 Lectures 1-12: Multi-model conversations (OpenAI + Claude together), LangChain vs LiteLLM, Gradio UI basics, streaming chatbot, system prompts and multi-shot prompting. Build the streaming chatbot in Gradio. Get comfortable with the UI layer.
3h	From-Scratch	P7-L07: GPT - Causal Language Modeling [CORE]. Understand how GPT generates text: autoregressive next-token prediction, causal masking, decoder-only architecture. Code: minimal GPT block, generation loop, temperature sampling. This is the architecture behind every model you called on Day 1.
2.5h	From-Scratch	P10-L01: Tokenizers - BPE, WordPiece, SentencePiece [CORE]. Understand what tiktoken does. Code: BPE training loop, merge rules, encode/decode. Understand why token != word and how this affects cost and context windows.

Deliverable: Streaming Gradio chatbot working. GPT causal LM architecture understood. BPE tokenizer code written. Can explain tokenization to a non-technical person.

Day 4

Time	Activity	Detail
3h	Ed Donners	W2 Lectures 13-24: Tool calling / function calling, airline AI assistant with tool calling + SQLite (Project 2), DALL-E 3 image generation, text-to-speech, multimodal AI, agentic AI intro. Build the airline assistant. This is your first agent - it calls tools (search flights, book seats).
3h	From-Scratch	P10-L02: Building a Tokenizer from Scratch [DEEP]. Go deeper than P10-L01: train a BPE tokenizer on a real text corpus, handle special tokens, chat templates. Compare your tokenizer output to tiktoken output on the same text.
2.5h	From-Scratch	P10-L06: Instruction Tuning - SFT [CORE] (skim). Understand what makes a chat model different from a base model. P10-L07: RLHF - Reward Model + PPO [DEEP] (skim). P10-L08: DPO [DEEP] (skim). Goal: understand the 3-step pipeline (pretrain -> SFT -> RLHF/DPO) at a conceptual level. You dont need to code these - just understand the pipeline.

Deliverable: Airline assistant with tool calling complete. Custom BPE tokenizer trained. Understand base vs chat vs reasoning model pipeline.

Day 5

Time	Activity	Detail
3h	Roadmap	R-P1 Setup: Read IMPLEMENTATION_PLAN.md for Project 1 (Advanced RAG). Create git repo: git init advanced-rag. Phase 1: Docker Compose setup (PostgreSQL 16 + pgvector). Database schema: documents, chunks, queries, golden_questions tables. Run the SQL from the plan. Verify pgvector extension is working with a test vector insert.
3h	Roadmap	R-P1 Phase 2: Document ingestion pipeline. Build: DocumentLoader (txt, md, pdf), ChunkingOrchestrator with 3 strategies (fixed_1000_200, semantic_spacy, sentence_window_3), EmbeddingGenerator (BGE-large-en-v1.5 via HuggingFace). Use AI to generate boilerplate. Verify: ingest 10 test documents, check chunks table has rows with embeddings.
2.5h	Roadmap	R-P1 Phase 2 continued: Test all 3 chunking strategies. Verify HNSW indexes are created for each strategy. Run EXPLAIN ANALYZE on a vector search query. Confirm retrieval works before moving to query pipeline.

Deliverable: R-P1 repo created. Docker Compose with PostgreSQL + pgvector running. 3 chunking strategies implemented. 10 documents ingested with embeddings. Vector search query returns results.

4. Phase 2: RAG Deep Dive (Days 6-15)

Ed Donners Weeks 3-5. The most intensive phase. HuggingFace, model evaluation, and the full RAG stack. Roadmap Projects 1-3 (Advanced RAG, Secure RAG, Advanced RAG Architectures).

Day 6

Time	Activity	Detail
3h	Ed Donners	W3 Lectures 1-12: HuggingFace platform tour, Google Colab + GPU setup [SKIP if you know Colab], HF Pipelines: sentiment, NER, Q&A, image classification, audio, diffusion. Run each pipeline type. Understand what pipeline() does under the hood.
3h	Roadmap	R-P1 Phase 3: Dual indexing. Build BM25 index (PostgreSQL tsvector) and vector index (pgvector HNSW). Implement hybrid retrieval: BM25 top-50 + Vector top-50 -> Reciprocal Rank Fusion (RRF) -> top-20 candidates. Code the RRF algorithm. Test: compare BM25-only vs vector-only vs hybrid on 5 queries.
2.5h	From-Scratch	P7-L07 review + P7-L12: KV Cache, Flash Attention and Inference Optimization [CORE]. Understand what model.generate() does under the hood. KV cache: why it speeds up generation. Flash Attention: why it reduces memory. This connects W3 inside LLaMA to real inference performance.

Deliverable: R-P1 hybrid retrieval working (BM25 + vector + RRF). HF Pipelines tested. Understand KV cache and Flash Attention at a conceptual level.

Day 7

Time	Activity	Detail
3h	Ed Donners	W3 Lectures 13-23: Tokenizers in action (encoding/decoding, chat templates, special tokens for LLaMA, Phi-4, DeepSeek, QWENCoder), transformers low-level API (model.generate, attention weights, quantization), inside LLaMA architecture (PyTorch model inspection), running open-source models (Phi, Gemma, Qwen, DeepSeek). Inspect a real models architecture in PyTorch.
3h	Roadmap	R-P1 Phase 4: Query transformation. Implement HyDE (generate hypothetical document, embed it, search) and Multi-Query expansion (3 reformulations of the query, embed each, search, merge results). Build the QueryTransform class with strategy parameter. Test on 5 queries: compare no-transform vs HyDE vs Multi-Query.
2.5h	From-Scratch	P10-L11: Quantization: INT8, GPTQ, AWQ, GGUF [CORE] (skim). Understand WHY Ollama can run models locally. 4-bit quantization = 70B model fits on a single GPU. Read the lesson, understand the trade-offs (quality vs memory vs speed).

Deliverable: R-P1 query transformation (HyDE + Multi-Query) working. LLaMA architecture inspected in PyTorch. Understand quantization trade-offs.

Day 8

Time	Activity	Detail
3h	Roadmap	R-P1 Phase 5: Cross-encoder re-ranking. Implement BGE-reranker-large: top-20 candidates -> top-5. Build the ReRanker class. Test: compare hybrid-only vs hybrid+rerank on 5 queries. Measure: does re-ranking improve relevance ordering? Document the latency cost.
3h	Roadmap	R-P1 Phase 6: LLM generation + FastAPI endpoint. Build: POST /query accepts strategy parameter (chunking + transform + rerank config), returns ranked results + LLM answer. Implement the full pipeline: query -> transform -> hybrid retrieve -> rerank -> LLM answer. Use AI to generate FastAPI boilerplate. Test the endpoint with curl.
2.5h	Roadmap	R-P1 Phase 7: Evaluation dashboard. Build Streamlit app: recall@k, MRR, nDCG, precision@k across all strategy combinations. Create a golden Q&A; dataset (10 questions with relevant chunk IDs). Run the evaluation runner. Verify: dashboard shows >15% nDCG variance between strategies. Write README.md. Commit. Mark R-P1 as complete in ROADMAP.md.

Deliverable: R-P1 COMPLETE. Cross-encoder re-ranking working. FastAPI /query endpoint deployed. Streamlit evaluation dashboard showing metrics. README written. Git committed.

Day 9

Time	Activity	Detail
3h	Ed Donners	W4 Lectures 1-10: Model evaluation, comparing frontier models on tasks, code generation (Python to C++, Project 4), reasoning models and reasoning effort. Build the code generation project. Understand: what makes a model good at a task? How do you measure it?
3h	Roadmap	R-P2 Setup + Phase 1: Read IMPLEMENTATION_PLAN.md for Project 2 (Secure RAG). Create git repo. Docker Compose (PostgreSQL + pgvector). Database schema: users, documents, chunks, audit_logs tables. Key difference from P1: customer_id (tenant isolation), allowed_roles (RBAC), audit_logs (append-only). Implement JWT auth middleware. Create test users for 2 tenants with different roles.
2.5h	From-Scratch	P10-L10: Evaluation - Benchmarks, Evals [CORE]. Understand how models are evaluated: MMLU, HumanEval, GSM8K, MT-Bench. Understand LLM-as-judge pattern. This connects Eds W4 model evaluation to the actual science behind it.

Deliverable: R-P2 repo created. JWT auth working. 2 test tenants with different roles. Understand model evaluation benchmarks. Code gen project complete.

Day 10

Time	Activity	Detail
3h	Ed Donners	W4 Lectures 11-21: Business task evaluation, model selection for production, agentic AI in action (Deep Research, Claude Code, Agent Mode), LLM competition game. Understand: model selection is a business decision, not just a technical one. Cost vs quality vs latency trade-offs.
3h	Roadmap	R-P2 Phase 2: Tenant isolation. Implement pre-filter vector search: WHERE customer_id = ? AND allowed_roles && user.roles before vector similarity. Implement Row-Level Security (RLS) as defense-in-depth. Test: verify zero cross-tenant leakage. Write automated test that tries to access other tenants data.
2.5h	From-Scratch	P5-L27: LLM Evaluation: RAGAS, DeepEval, G-Eval [CORE]. Understand the evaluation frameworks that measure RAG quality: faithfulness, answer relevance, context precision/recall. This is the theory behind R-P1s evaluation dashboard and R-P7s eval harness.

Deliverable: R-P2 tenant isolation + RLS working. Zero cross-tenant leakage verified by test. Understand RAGAS/DeepEval evaluation frameworks.

Day 11

Time	Activity	Detail
3h	Ed Donners	W5 Lectures 1-10: RAG big idea, vector data stores, vectors and LangChain, vector databases (Chroma, FAISS), embeddings and embedding models, t-SNE visualization of embeddings. Build Eds basic RAG pipeline. See how ChromaDB works. Visualize embeddings with t-SNE.
3h	Roadmap	R-P2 Phase 3: Citation tracking + audit logging. Every answer includes structured citations (doc_path, chunk_id, snippet). Implement audit_logs table with append-only triggers (prevent UPDATE/DELETE). Log every query: user_id, customer_id, question, retrieved_chunk_ids, answer, citations, timestamp. Build the audit log query API (filter by tenant, user, time range).
2.5h	From-Scratch	P11-L06: RAG: Retrieval-Augmented Generation [CORE]. Build RAG from first principles (no LangChain). Code: embed query, vector search, build context, call LLM with context. Compare your from-scratch RAG to Eds LangChain RAG. Understand what LangChain adds (and what it hides).

Deliverable: R-P2 citations + audit logging working. Append-only triggers tested. From-scratch RAG (no framework) built and compared to LangChain version.

Day 12

Time	Activity	Detail
3h	Ed Donners	W5 Lectures 11-20: Document chunking, text splitters, RAG pipeline with LangChain + Chroma, RAG with conversation history, Gradio UI for RAG. Build Eds full RAG pipeline. See how chunking affects retrieval quality.
3h	Roadmap	R-P2 Phase 4: Finish + polish. Build FastAPI endpoints: POST /upload (with permission check), POST /query (with tenant filter + citations + audit). Write README.md with architecture diagram. Docker Compose up should start the full system. Write tests: tenant isolation, RBAC, audit log immutability. Mark R-P2 as complete in ROADMAP.md.

Time	Activity	Detail
2.5h	From-Scratch	P5-L23: Chunking Strategies for RAG [CORE] + P11-L07: Advanced RAG: Chunking, Reranking [CORE]. Understand: fixed-size, semantic, sentence-window chunking. When to use each. Re-ranking: bi-encoder vs cross-encoder. Why re-ranking improves precision. This is the theory behind R-P1s chunking and re-ranking implementations.

Deliverable: R-P2 COMPLETE. FastAPI endpoints working. README written. Tests passing. Git committed. Understand chunking strategies and re-ranking theory.

Day 13

Time	Activity	Detail
3h	Ed Donners	W5 Lectures 21-32: RAG evaluation (MRR, NDCG, LLM-as-judge, golden data), advanced RAG (query rewriting, re-ranking, query expansion, GraphRAG), semantic chunking, production RAG. This is Eds most RAG-intensive content. Take detailed notes on evaluation metrics.
3h	Roadmap	R-P3 Setup + Phase 1: Read IMPLEMENTATION_PLAN.md for Project 3 (Advanced RAG Architectures). Create git repo. Docker Compose (PostgreSQL + pgvector + Neo4j). Phase 1: GraphRAG pipeline. LLM entity extraction from text -> entities + relationships -> Neo4j graph construction. Install Neo4j Python driver. Write entity extraction prompt. Test on 5 documents.
2.5h	From-Scratch	P5-L14: Information Retrieval and Search [CORE]. Understand the IR theory behind vector DBs: boolean retrieval, TF-IDF, BM25, vector space model, cosine similarity. This is the math behind what Chroma/pgvector/FAISS do. Code: implement BM25 from scratch. Compare to PostgreSQLs tsvector ranking.

Deliverable: R-P3 repo created. Neo4j running. Entity extraction working on test documents. BM25 implemented from scratch. IR theory understood.

Day 14

Time	Activity	Detail
3h	Ed Donners	W5: Knowledge worker project (Project 5) + buffer. Finish any incomplete W5 content. Build Eds knowledge worker - a RAG system over a real document set. This is your chance to see a production-quality RAG system built by an expert.
3h	Roadmap	R-P3 Phase 2: Agentic RAG. Build iterative retrieval loop: LLM decides what to retrieve, assesses if context is sufficient, retrieves again if not (max 5 iterations). Implement Self-RAG reflection tokens: [RETRIEVE]/[NO RETRIEVE], [RELEVANT]/[IRRELEVANT]. Build the decision framework: vanilla RAG vs GraphRAG vs Agentic RAG vs Multimodal RAG. Implement the Streamlit comparison dashboard.
2.5h	From-Scratch	P5-L22: Embedding Models Deep Dive [CORE]. Understand how embedding models are trained: contrastive learning, triplet loss, hard negative mining. Why BGE > OpenAI embeddings for some tasks. This is the theory behind your embedding choices in R-P1 and R-P2.

Deliverable: R-P3 Agentic RAG loop working. Decision framework implemented. Streamlit comparison dashboard running. Understand embedding model training. Knowledge worker project complete.

Day 15

Time	Activity	Detail
3h	Roadmap	R-P3 Phase 3: Multimodal RAG. Install CLIP (openai/clip-vit-base-patch32). Embed images with CLIP (512-dim) alongside text with BGE (1024-dim). Store both in pgvector (separate vector dimensions). Implement cross-modal retrieval: text query -> image results, image query -> text results. Extract images from PDFs with PyMuPDF. Test on documents with embedded charts/diagrams.
3h	Roadmap	R-P3 Phase 4: Finish. Build FastAPI comparison API: POST /query with architecture parameter (vanilla, graph, agentic, multimodal). Write KAG/LightRAG comparison table (survey, not implementation). Write README.md with architecture decision framework. Docker Compose up. Mark R-P3 as complete in ROADMAP.md.
2.5h	Buffer	Buffer: Re-do any weak project from Phase 2. Re-test R-P1 evaluation dashboard. Verify all 3 repos have clean git history, READMEs, and Docker Compose files. Practice explaining each projects architecture out loud (interview prep).

Deliverable: R-P3 COMPLETE. Multimodal RAG with CLIP working. Comparison API deployed. README written. All 3 RAG projects (P1, P2, P3) complete with Docker, tests, and documentation.

5. Phase 3: Agents (Days 16-24)

Ed Donners Weeks 6-8. Traditional ML baselines (skim - you know these), fine-tuning GPT-4o-mini, QLoRA on LLaMA, and the multi-agent capstone. Roadmap Projects 4-6 (Data Analysis Agent, Document Pipeline, Multi-Agent Systems).

Day 16

Time	Activity	Detail
2h	Ed Donners	W6 Lectures 1-10: Datasets, Amazon data curation, Hugging Face datasets, weighted sampling, 5-step AI process. [SKIP traditional ML baselines - you know Random Forest/XGBoost from data engineering]. Focus on: dataset distribution analysis, deduplication, HF datasets API.
3h	Roadmap	R-P4 Setup + Phase 1: Read IMPLEMENTATION_PLAN.md for Project 4 (Data Analysis Agent). Create git repo. Docker Compose (PostgreSQL with Northwind sample data). Create read-only DB user: data_agent_reader (SELECT only, statement_timeout=30s, work_mem=64MB). Build function calling setup: define tools (get_schema, execute_sql, generate_chart) as OpenAI function schemas. Test: LLM calls get_schema and receives table/column information.
3.5h	From-Scratch	P3-L03: Backpropagation from Scratch [CORE] (if you havent written it by hand before). Code: forward pass, backward pass, gradient computation, weight update. Understand: chain rule, why gradients flow backwards, why ReLU helps. If you HAVE done this before, skip to P3-L05: Loss Functions: MSE, Cross-Entropy, Contrastive [CORE]. Understand cross-entropy: the loss function used in every LLM fine-tuning job youll see in W7.

Deliverable: R-P4 repo created. Northwind data loaded. Read-only DB user configured. Function calling tools defined. Backprop or loss functions understood from scratch.

Day 17

Time	Activity	Detail
3h	Ed Donners	W6 Lectures 11-20: Fine-tuning GPT-4o-mini with OpenAI SFT API, when fine-tuning fails, neural networks in PyTorch, testing frontier models on price prediction (Project 6). Build Eds fine-tuning job. See the training data format, the API call, the evaluation. Understand: fine-tuning is a data problem, not a code problem.
3h	Roadmap	R-P4 Phase 2: SQL validation layer. Implement: sqlglot parse (syntax check), safety check (no DDL/DML, read-only), LIMIT enforcement (inject LIMIT if missing), EXPLAIN cost estimation (reject if too expensive). Build the ReAct agent loop: Reason -> Act (call tool) -> Observe -> Reason. Test: agent writes SQL for top 10 customers by revenue, validates, executes, returns results.
2.5h	Roadmap	R-P4 Phase 3: Error recovery loop. If SQL fails, feed error back to LLM -> regenerate -> retry (max 3). Build conversation memory (in-memory dict + SQLite/Postgres persistence). Test multi-turn: show me top customers -> now filter by Germany (agent reuses context). Write README.md. Mark R-P4 as complete in ROADMAP.md.

Deliverable: R-P4 COMPLETE. SQL validation layer working. ReAct agent loop with error recovery. Multi-turn conversation memory. README written. GPT-4o-mini fine-tuning understood.

Day 18

Time	Activity	Detail
3h	Ed Donners	W6 Lectures 21-27: Deep neural network redemption (Project 6 continued), MLOps, Groq batch processing, baseline models vs LLM comparison. Understand: when does fine-tuning beat prompt engineering? When does traditional ML beat LLMs? Key insight: for structured/tabular data, XGBoost often beats LLMs. Know when to use which.
3h	Roadmap	R-P5 Setup + Phase 1: Read IMPLEMENTATION_PLAN.md for Project 5 (Document Processing Pipeline). Create git repo. Docker Compose (PostgreSQL + Airflow). Build ingestion layer: parsers for PDF (pypdf), DOCX (python-docx), TXT, EML (mailparser), images (Tesseract OCR). Build LLM classifier: classify as invoice/contract/email/receipt/other with confidence score. Test: classify 10 mixed documents. Verify confidence scores are reasonable.
2.5h	Roadmap	R-P5 Phase 2: Structured extraction. Build Pydantic v2 schemas for each document type (InvoiceSchema, ContractSchema, EmailSchema, ReceiptSchema). Use OpenAI structured output: client.beta.chat.completions.parse() with Pydantic response_format. Test: extract fields from 5 invoices. Verify Pydantic validation catches bad data.

Deliverable: R-P5 repo created. Ingestion + classification working. Pydantic schemas defined. Structured extraction with OpenAI response_format tested.

Day 19

Time	Activity	Detail
3h	Ed Donners	W7 Lectures 1-10: QLoRA intro, LoRA training LLaMA 3.2, LoRA hyperparameters, 4-bit/8-bit quantization explained. Understand: LoRA = low-rank adaptation (freeze base weights, train small adapter). QLoRA = 4-bit base + LoRA adapter. See the training loop in Eds notebook.
3h	Roadmap	R-P5 Phase 3: Validation + retry. Implement: Pydantic validation + business rule validators (invoice total = sum of line items, date plausibility, tax rate ranges). Retry with error feedback: append validation errors to prompt, re-extract (max 2 retries). Build routing: validated records -> PostgreSQL, original files -> document store, audit log entry. Build Airflow DAG: scan -> ingest -> classify -> extract -> validate -> route. Test: run DAG on 20 documents. Verify audit log has entry for each.
2.5h	From-Scratch	P11-L08: Fine-Tuning with LoRA and QLoRA [CORE]. Understand the math behind what Ed showed you: low-rank decomposition ($W = W_0 + BA$ where B and A are small), gradient flow through frozen weights, why LoRA is parameter-efficient. Code: minimal LoRA layer in PyTorch.

Deliverable: R-P5 validation + retry + routing working. Airflow DAG running. LoRA math understood. Minimal LoRA layer coded.

Day 20

Time	Activity	Detail
3h	Ed Donners	W7 Lectures 11-20: Dataset prep for fine-tuning, base vs chat models, training hyperparameters (learning rate, optimizers, batch size), Weights & Biases, TRL SFT Trainer. See Eds full training run. Understand: learning rate is the most important hyperparameter. W&B; tracks loss curves, gradient norms, learning rate schedule.
3h	Roadmap	R-P5 Phase 4: Human review queue. Build Streamlit review UI: view original doc + extracted data + validation errors + confidence. Reviewer can edit fields, approve (routes to downstream), or reject (logs + archives). Write README.md. Docker Compose up (Airflow + Postgres + API + Streamlit). Mark R-P5 as complete in ROADMAP.md.
2.5h	Roadmap	R-P6 Setup: Read IMPLEMENTATION_PLAN.md for Project 6 (Multi-Agent Systems). Create git repo. Docker Compose (PostgreSQL + Redis). Install LangGraph. Build the StateGraph TypedDict: query, query_type, research_notes, draft, critic_feedback, revision_count, final_output, cost_log, messages, memory_context, status. Define the graph topology: router -> research -> writer -> critic -> synthesizer.

Deliverable: R-P5 COMPLETE. Human review queue in Streamlit. README written. R-P6 repo created. LangGraph StateGraph defined. Graph topology designed.

Day 21

Time	Activity	Detail
3h	Ed Donners	W7 Lectures 21-27: Monitoring training loss, overfitting, checkpoint selection, cross-entropy loss, full dataset training on A100, testing fine-tuned model vs GPT-4o (Project 7). See Eds full QLoRA training run on A100. Understand: training loss vs validation loss, when to stop training, how to select the best checkpoint.
3h	Roadmap	R-P6 Phase 2: Router agent + research agents. Build Router Agent: classifies query as technical/creative/analytical. Build 3 Research Agents (one per type) with tools: web_search_stub, knowledge_base_lookup. Implement conditional edges in LangGraph: router -> appropriate research node. Test: submit queries of each type, verify routing.
2.5h	Roadmap	R-P6 Phase 3: Writer + Critic agents. Writer Agent: produces draft from research notes. Critic Agent: reviews draft -> structured feedback {strengths, weaknesses, suggestions, approved: bool}. Implement revision loop: if not approved AND revision_count < 3 -> back to writer. Test: submit a query, watch the write-critique-revise cycle. Verify loop terminates.

Deliverable: R-P6 Router + Research + Writer + Critic agents working. Revision loop with max 3 rounds. LangGraph conditional routing tested. QLoRA training process understood.

Day 22

Time	Activity	Detail
3h	Ed Donners	W8 Lectures 1-11: Agentic AI intro, Modal serverless deployment, designing agent architectures, Modal platform setup. See how Ed deploys an agent to Modal. Understand serverless AI: cold starts, GPU provisioning, cost-per-invocation. [Note: your roadmap uses Docker, not Modal - but understand the serverless pattern].
3h	Roadmap	R-P6 Phase 4: Orchestrator-Worker pattern. Build Orchestrator Agent: decomposes complex task into subtasks, delegates to specialist workers, synthesizes results. Build Map-Reduce parallel agents: fan out >= 3 subtasks concurrently, reduce/synthesize. Build Supervisor pattern: monitors worker status, handles failures, routes to fallback. Test each pattern independently via API pattern parameter.
2.5h	Roadmap	R-P6 Phase 5: Agent memory + cost tracking. Short-term memory: Redis buffer (last 10 messages), summarization after 10 turns. Long-term memory: pgvector episodic memory (embed key facts). Cost tracker: per-agent token usage, total pipeline cost, latency per node. Build Streamlit dashboard: agent graph visualization + cost/latency breakdowns. Write README.md. Mark R-P6 as complete in ROADMAP.md.

Deliverable: R-P6 COMPLETE. Orchestrator-Worker + Map-Reduce + Supervisor patterns working. Agent memory (Redis + pgvector). Cost tracking. Streamlit dashboard. README written.

Day 23

Time	Activity	Detail
3h	Ed Donners	W8 Lectures 12-22: Multi-agent system (Project 8), autonomous deal-spotting system. Build Eds final project. This is the capstone of the Ed Donners course - a multi-agent system that autonomously finds and reports deals. See how all 8 weeks of skills come together.
3h	From-Scratch	P14-L01: The Agent Loop [CORE]. Understand the fundamental agent pattern: perceive -> reason -> act -> observe -> repeat. Code: minimal agent loop in Python. P14-L02: ReWOO and Plan-and-Execute [CORE]. Understand: plan all steps first, then execute. P14-L12: Anthropic Workflow Patterns [CORE]. The canonical patterns: prompt chaining, routing, parallelization, orchestrator-worker, evaluator-optimizer.
2.5h	From-Scratch	P14-L25: Multi-Agent Debate and Collaboration [CORE]. P14-L28: Orchestration Patterns - Supervisor, Swarm, Hierarchical [CORE]. Understand the patterns behind R-P6. Map each pattern to what you built. This is the theory that makes your R-P6 code make sense.

Deliverable: Ed Donners course COMPLETE (all 8 weeks, all 8 projects). Agent loop, plan-execute, workflow patterns, multi-agent orchestration understood from first principles.

Day 24

Time	Activity	Detail
4h	Buffer	Buffer: Re-do any weak project from Phase 3. Verify R-P4, R-P5, R-P6 all have: clean git history, READMEs, Docker Compose files, tests passing. Practice explaining each projects architecture out loud. Key interview question: Why did you use LangGraph instead of CrewAI? Answer: typed state graph, conditional edges, checkpoint support, better debugging.
4.5h	Buffer	Buffer continued: Polish all 6 completed roadmap projects. Update ROADMAP.md progress table (P1-P6 should be marked complete). Ensure each repo has: architecture diagram in README, metrics section, Docker Compose up works. Take a break - you have completed Ed Donners + 6 roadmap projects + all from-scratch [CORE] lessons.

Deliverable: All 6 roadmap projects (P1-P6) polished and complete. ROADMAP.md updated. Ed Donners course fully completed. Ready for Phase 4 (evaluation + infrastructure).

Milestone: Halfway through the plan. 6 portfolio projects + 8 Ed Donners projects + transformers/tokenizers/RAG/agents built from scratch.

6. Phase 4: Evaluation + Infrastructure (Days 25-34)

Ed Donners is complete. Full focus on AI-Engineering Roadmap Projects 7-10: evaluation harness, prompt A/B testing, LLM serving gateway, PII guardrails. This is the tier that separates AI engineers from API callers.

Day 25

Time	Activity	Detail
4h	Roadmap	R-P7 Setup + Phase 1: Read IMPLEMENTATION_PLAN.md for Project 7 (LLM Evaluation Harness). Create git repo. Docker Compose (PostgreSQL). Database schema: prompt_versions, eval_datasets, eval_runs, eval_case_results, human_feedback tables. Build eval dataset format: YAML files with test cases (question, expected_answer, relevant_contexts, metadata). Create 20 test cases for a customer support FAQ scenario. Build the System Under Test (SUT) adapter: pluggable interface that calls any RAG system.
4.5h	Roadmap	R-P7 Phase 2: Deterministic metrics. Implement: latency (p50, p95), cost (token count * price), context precision (fraction of retrieved context that is relevant), context recall (fraction of relevant context that is retrieved), citation accuracy (fraction of citations that match source chunks). Unit test each metric with known-good and known-bad fixtures. Build the Eval Runner: for each case, call SUT -> compute metrics -> assemble CaseResult.

Deliverable: R-P7 repo created. Database schema deployed. 20 eval test cases in YAML. 5 deterministic metrics implemented and unit tested. Eval Runner working.

Day 26

Time	Activity	Detail
4h	Roadmap	R-P7 Phase 3: LLM-as-judge metrics. Implement faithfulness (is the answer grounded in the context?) and answer relevance (does the answer address the question?). Use GPT-4 as judge, GPT-4o-mini as SUT (judge must be stronger than SUT). Build structured judge output: JSON with score (0-1) + reasoning. Calibrate: run judge on 50 cases, compute inter-rater agreement (Cohens kappa) vs human gold labels. Target: kappa >= 0.6.
4.5h	Roadmap	R-P7 Phase 4: Context engineering metrics. Implement: context utilization (fraction of retrieved context actually used in the answer), compression ratio (tokens after compression / before), position bias (detect lost in middle effect), context overflow (retrieved context exceeded model window). Build Streamlit eval dashboard: metric trends over time, per-prompt-version comparisons, failing test cases. Build CI integration: pytest with @pytest.mark.eval, GitHub Actions workflow that runs eval suite on every PR. Write README.md. Mark R-P7 as complete in ROADMAP.md.

Deliverable: R-P7 COMPLETE. LLM-as-judge with GPT-4 working. Context engineering metrics implemented. Streamlit eval dashboard running. CI integration with GitHub Actions. README written.

Day 27

Time	Activity	Detail
4h	Roadmap	R-P8 Setup + Phase 1: Read IMPLEMENTATION_PLAN.md for Project 8 (Prompt Versioning + A/B Testing). Create git repo. Docker Compose (PostgreSQL). Database schema: prompts, prompt_versions (git-like versioning, status: draft/staging/production/archived), eval_datasets, eval_items, ab_tests, test_results, metric_history, promotions tables. Build prompt store: CRUD + versioning. Jinja2 templating with SandboxedEnvironment. Variable validation: StrictUndefined raises on missing variables. Test: create a prompt, create 3 versions, verify version history is queryable.
4.5h	Roadmap	R-P8 Phase 2: A/B test runner. Given a prompt_id with 2+ versions: load both versions, render with eval dataset variables, call LLM for each, measure latency_ms + token_count + output_length, call LLM-as-judge to compare output to expected output -> score (0-1). Compute per-variant aggregate metrics: mean_accuracy, mean_latency, mean_cost. Test: run A/B test on 2 prompt versions with 20 eval items. Verify results are stored in DB.

Deliverable: R-P8 repo created. Prompt versioning with Jinja2 working. A/B test runner implemented. First A/B test executed with results in database.

Day 28

Time	Activity	Detail
4h	Roadmap	R-P8 Phase 3: Statistical significance. Compute: win-rate (variant wins on accuracy per item), Wilson confidence interval on win-rate, two-proportion z-test p-value, required sample size for desired power. Declare winner ONLY if $p < 0.05$ AND sample size adequate. Use scipy + statsmodels for statistical computations. Build promotion/rollback API: POST /promote (atomic DB transaction: archive old production, set new production, insert promotion record). Build Python SDK: PromptClient.get_production_prompt(summarizer) returns rendered template.
4.5h	Roadmap	R-P8 Phase 4: Streamlit dashboard. Visualize: win-rate chart, metric comparison table, sample outputs side-by-side, prompt version history, metric trends over time (line charts). Build promotion/rollback UI: click to promote or roll back a version. Write README.md. Docker Compose up (API + DB + dashboard). Mark R-P8 as complete in ROADMAP.md.

Deliverable: R-P8 COMPLETE. Statistical significance testing working. Promotion/rollback API + SDK. Streamlit dashboard with A/B test visualization. README written.

Day 29

Time	Activity	Detail
4h	Roadmap	R-P9 Setup + Phase 1: Read IMPLEMENTATION_PLAN.md for Project 9 (LLM Serving Gateway). Create git repo. Docker Compose (PostgreSQL + Redis). Database schema: cost_records, tenants, budget_usage, model_pricing tables. Build FastAPI gateway: auth + tenant resolve -> rate limiter (Redis sliding window) -> budget check (per-tenant monthly token budget) -> semantic cache (Redis vector index). Implement semantic cache: embed prompt with text-embedding-3-small, search Redis for cosine sim ≥ 0.95 , return cached response if hit. Test: send same query twice, verify second is cache hit.
4.5h	Roadmap	R-P9 Phase 2: Model routing. Implement: task_type detection (simple vs complex), simple -> GPT-4o-mini, complex -> GPT-4o. Confidence check: if mini_response.confidence < 7 , escalate to GPT-4o. Build provider failover chain: OpenAI -> Anthropic -> Ollama (local). Test: simulate OpenAI 5xx, verify automatic failover to Anthropic. Verify: zero downtime during simulated primary outage.

Deliverable: R-P9 repo created. Semantic cache working ($\geq 50\%$ hit rate on repeated queries). Model routing with confidence escalation. Provider failover chain tested.

Day 30

Time	Activity	Detail
4h	Roadmap	R-P9 Phase 3: Cost tracking + structured logging. Every request: log tenant, model, tokens (in/out), latency, cache hit/miss, cost. Insert into cost_records table. Update budget_used in Redis. Build daily cost report: SELECT tenant, date, SUM(cost) FROM cost_records GROUP BY tenant, date. Verify matches provider invoices within $\pm 2\%$. Build admin dashboard (Streamlit): cache hit rate, cost per tenant, model distribution, budget usage, rate-limit stats. Live-updating from PostgreSQL.
4.5h	Roadmap	R-P9 Phase 4: Polish + finish. Rate limiting: sliding-window in Redis (per-minute, per-hour, per-day). Over-limit returns HTTP 429 with Retry-After header. Test: concurrent burst, verify no burst-through. Write README.md with the cost math from the plan (87% reduction example). Docker Compose up (gateway + Redis + Postgres + Streamlit). Mark R-P9 as complete in ROADMAP.md.

Deliverable: R-P9 COMPLETE. Cost tracking + structured logging. Admin dashboard running. Rate limiting tested. README with cost math. 87% cost reduction demonstrated.

Day 31

Time	Activity	Detail
4h	Roadmap	R-P10 Setup + Phase 1: Read IMPLEMENTATION_PLAN.md for Project 10 (PII Redaction Guardrails). Create git repo. Docker Compose (PostgreSQL). Database schema: tenants, tenant_config, allowlist, redaction_events, audit_logs tables. Build FastAPI proxy middleware: client sets base_url to proxy, proxy intercepts every LLM API call. Layer 1: Regex patterns. Implement: email, phone, SSN, IBAN, credit card, Steuer-ID, PLZ, German address. Test: scan My name is Anna Mueller, email: a.mueller@gmx.de, Steuer-ID: 12345678901 -> verify all 3 PII types detected.
4.5h	Roadmap	R-P10 Phase 2: Layer 2 - NER. Install Microsoft Presidio + spaCy. Load en_core_web_lg + de_core_news_lg for German NER. Detect: person names, organizations, locations, dates, addresses. Test: scan same text -> verify Anna Mueller (PERSON), Kiel (LOCATION), Hauptstr. 42 (ADDRESS) detected. Build merge + deduplicate: combine Layer 1 + Layer 2 results, resolve overlapping spans. Apply tenant allowlist: remove allowlisted entities from detection list.

Deliverable: R-P10 repo created. Proxy middleware working. Layer 1 (regex) + Layer 2 (NER) detection. Merge/dedupe + tenant allowlist. German PII patterns (Steuer-ID, PLZ) detected.

Day 32

Time	Activity	Detail
4h	Roadmap	R-P10 Phase 3: Layer 3 - LLM contextual + redaction engine. LLM contextual: use gpt-4o-mini to catch implicit PII (my boss is the CEO -> PERSON). Only runs if Layer 1+2 found fewer entities than expected. Redaction engine: per PII type, per tenant, apply strategy (mask/hash/synthetic). Mask: [REDACTED_PERSON]. Hash: [HASH_a3f9...]. Synthetic: Faker with de_DE locale. Build output scanner: same 3-layer pipeline on LLM response (post-LLM). Block or sanitize leaked PII in output. Log every redaction event to compliance log.
4.5h	Roadmap	R-P10 Phase 4: Compliance + finish. Audit log: append-only (DB trigger blocks UPDATE/DELETE). Build Streamlit compliance dashboard: redaction statistics, PII type distribution, compliance report export (CSV/PDF). Build GDPR compliance mapping table (Appendix E of the plan). Write README.md. Docker Compose up (proxy + DB + dashboard). Test: full end-to-end: send prompt with PII -> verify redacted prompt reaches LLM -> verify response has no leaked PII -> verify audit log has entry. Mark R-P10 as complete in ROADMAP.md.

Deliverable: R-P10 COMPLETE. 3-layer PII detection (regex + NER + LLM). Redaction engine with mask/hash/synthetic. Compliance audit log (append-only). Streamlit dashboard. GDPR mapping. README written.

Day 33

Time	Activity	Detail
4.5h	Buffer	Buffer: Re-do any weak project from Phase 4. Verify R-P7, R-P8, R-P9, R-P10 all have: clean git history, READMEs, Docker Compose files, tests passing. Key interview prep: How do you evaluate a RAG system? -> describe R-P7s metrics (faithfulness, relevance, context precision/recall, citation accuracy). How do you control LLM costs? -> describe R-P9s semantic cache + model routing + budget enforcement.
4h	Buffer	Buffer continued: Practice explaining each projects architecture decision. Why pgvector over Qdrant? (single infra, SQL-level filtering, sufficient scale). Why Presidio + spaCy? (industry standard, multilingual, contextual entity linking). Why GPT-4 as judge, GPT-4o-mini as SUT? (judge must be stronger than system under test).

Deliverable: All 4 Phase 4 projects (P7-P10) polished and complete. Interview prep for evaluation, cost control, and PII questions done.

Day 34

Time	Activity	Detail
4.5h	Buffer	Buffer: Polish all 10 completed roadmap projects. Update ROADMAP.md progress table (P1-P10 should be marked complete). Ensure each repo has: architecture diagram in README, metrics section, Docker Compose up works, tests pass. Run docker compose up in each repo to verify everything still works.
4h	Buffer	Buffer continued: Commit any uncommitted changes. Clean up git history. Write a 1-paragraph summary of each project for your portfolio. Take a break - tomorrow you start the capstone (Project 11), which combines everything.

Deliverable: All 10 roadmap projects (P1-P10) polished. ROADMAP.md updated. All Docker Compose files verified. Git history clean. Ready for capstone.

Milestone: 10 of 12 roadmap projects complete. Only the capstone (P11) and case study (P12) remain.

7. Phase 5: Capstone (Days 35-45)

Project 11 (Customer Support AI System) combines all 10 previous projects into one system. Project 12 (Technical Case Study) documents everything for your portfolio. This is the phase that makes you job-ready.

Day 35

Time	Activity	Detail
4h	Roadmap	R-P11 Setup + Phase 1: Read IMPLEMENTATION_PLAN.md for Project 11 (Customer Support AI System). Create git repo. Docker Compose (PostgreSQL+pgvector + Redis + Neo4j + Airflow). Database schema: tickets, ticket_classifications, responses, response_evaluations, cost_records, audit_logs, pii_redactions, human_reviews tables. Build ticket ingestion API: POST /tickets accepts {customer_id, subject, body, channel}. Insert into tickets table. Build the ticket router: routes to the processing pipeline.
4.5h	Roadmap	R-P11 Phase 2: PII redaction layer. Reuse R-P10s 3-layer detection. Every incoming ticket: scan for PII -> redact -> store redacted version + redaction events. Build the classification layer: LLM classifies ticket on 3 dimensions: urgency (low/medium/high/critical), category (billing/technical/account/general), sentiment (positive/neutral/frustrated/angry). Store classification in ticket_classifications table. Test: ingest 10 sample tickets.

Deliverable: R-P11 repo created. Docker Compose with all services running. Ticket ingestion API working. PII redaction + classification layers operational.

Day 36

Time	Activity	Detail
4.5h	Roadmap	R-P11 Phase 3: RAG knowledge base. Reuse R-P1s hybrid retrieval (BM25 + pgvector + RRF). Re-use R-P1s chunking strategies and re-ranking. Ingest knowledge base articles (FAQ, product docs, policies) into the knowledge base. Build the retrieval function: given ticket subject + body, retrieve top-5 relevant KB articles. Test: submit a billing question -> verify retrieval returns billing-related articles.
4h	Roadmap	R-P11 Phase 4: Response generation. Build the response drafter: given ticket + retrieved KB context + classification, generate a draft response. Use a response template: greeting + acknowledge issue + solution from KB + next steps + signature. Generate confidence score: LLM self-assessment (0-1) + retrieval confidence (cosine sim of top result). If confidence >= 0.8: auto-respond. If < 0.8: route to human review queue. Test: submit 10 tickets, verify auto-respond vs human-review split is reasonable.

Deliverable: R-P11 RAG knowledge base working (reuses R-P1). Response generation with confidence scoring. Auto-respond vs human-review routing tested.

Day 37

Time	Activity	Detail
4.5h	Roadmap	R-P11 Phase 5: Evaluation layer. Reuse R-P7s eval harness. Evaluate every auto-generated response: faithfulness (grounded in KB?), answer relevance (addresses ticket?), citation accuracy (citations match sources?). Store evaluation scores in response_evaluations table. Build the feedback loop: if faithfulness < 0.7, flag for human review even if confidence was high. Build the human review queue (Streamlit): reviewer sees ticket + draft + eval scores + KB context. Reviewer can edit, approve, or reject. Approved responses are sent.
4h	Roadmap	R-P11 Phase 6: Cost tracking + audit. Reuse R-P9s cost tracking. Log every LLM call: ticket_id, model, tokens, cost. Build cost dashboard: cost per ticket, cost per category, monthly trend. Reuse R-P2s audit logging pattern: every action logged (ingest, redact, classify, retrieve, generate, evaluate, auto-respond, human-review, send). Audit log is append-only. Build audit query API: filter by ticket, customer, action, time range.

Deliverable: R-P11 evaluation layer working (reuses R-P7). Human review queue in Streamlit. Cost tracking (reuses R-P9). Audit logging (reuses R-P2).

Day 38

Time	Activity	Detail
4.5h	Roadmap	R-P11 Phase 7: End-to-end integration. Wire the full pipeline: ticket ingest -> PII redact -> classify -> RAG retrieve -> draft response -> confidence score -> eval -> auto-respond or human-review -> cost track -> audit log. Build the Streamlit customer support dashboard: ticket queue, processing status, auto-responded vs human-reviewed stats, cost breakdown, eval score trends. Test end-to-end: submit 20 diverse tickets (billing, technical, account, general). Verify: all 20 processed, PII redacted, classified, responses generated, costs tracked, audit logged.
4h	Roadmap	R-P11 Phase 8: Polish. Write README.md with full architecture diagram. Document the 14 component-to-project mappings (which R-P# is reused where). Docker Compose up should start the entire system. Write integration tests. Mark R-P11 as complete in ROADMAP.md.

Deliverable: R-P11 COMPLETE. Full end-to-end pipeline working. Streamlit dashboard running. 20 tickets processed end-to-end. README with architecture + component mappings. Integration tests passing.

This is the capstone project. It demonstrates you can architect and build a production AI system.

Day 39

Time	Activity	Detail
4.5h	Roadmap	R-P12 Phase 1: Read IMPLEMENTATION_PLAN.md for Project 12 (Technical Case Study). Write CASE_STUDY.md (15-20 pages). Structure: Executive Summary, System Architecture, Component Design (one section per component: ingestion, PII, classification, RAG, generation, evaluation, human review, cost tracking, audit), Data Flow Diagrams, Technology Decisions, Performance & Metrics, Failure Modes & Mitigations, Operational Runbook, Lessons Learned. Write the first 8 sections (Executive Summary through Component Design).
4h	Roadmap	R-P12 Phase 2: Write 5 Architecture Decision Records (ADRs). ADR-001: Why pgvector over Qdrant. ADR-002: Why LangGraph over CrewAI. ADR-003: Why GPT-4o-mini as default model. ADR-004: Why Presidio + spaCy for PII. ADR-005: Why Streamlit for dashboards. Each ADR: Context, Decision, Consequences, Alternatives Considered. Write the metrics tables: latency by component, cost per ticket, eval scores by category, cache hit rate, auto-respond rate.

Deliverable: R-P12: CASE_STUDY.md first 8 sections written. 5 ADRs written. Metrics tables compiled.

Day 40

Time	Activity	Detail
4.5h	Roadmap	R-P12 Phase 3: Write the remaining CASE_STUDY.md sections: Failure Modes & Mitigations (LLM hallucination, PII leakage, cost overrun, eval false positives, RAG retrieval failure, agent loop infinite recursion, model outage, cache poisoning). Operational Runbook (deployment, monitoring, incident response, rollback procedures). Lessons Learned (what worked, what didnt, what youd do differently). Review the full document. Target: 15-20 pages, clear, professional.
4h	Roadmap	R-P12 Phase 4: Build presentation deck outline. 15-slide structure: Title, Problem Statement, Solution Architecture, Component Deep-Dive (3-4 slides), Live Demo Screenshots, Metrics & Results, Technology Stack, Challenges & Solutions, Future Improvements, Q&A.; Write speaker notes for each slide. Mark R-P12 as complete in ROADMAP.md.

Deliverable: R-P12: CASE_STUDY.md complete (15-20 pages). Presentation deck outline with speaker notes. R-P12 COMPLETE. All 12 roadmap projects done.

Day 41

Time	Activity	Detail
4.5h	Buffer	Final polish: Review all 12 roadmap project repos. Ensure each has: clean git history, README with architecture diagram, Docker Compose up works, tests pass. Update ROADMAP.md: all 12 projects marked complete. Create a master portfolio README that links to all 12 projects with 1-paragraph descriptions. Push all repos to GitHub (if not already pushed).
4h	Buffer	Interview prep: Practice explaining each project in 2 minutes. Key questions to prepare: Walk me through your RAG architecture (R-P1+P2+P3). How do you evaluate LLM systems? (R-P7). How do you handle PII? (R-P10). How do you control costs? (R-P9). Describe your multi-agent system (R-P6). Walk me through your capstone (R-P11).

Deliverable: All 12 repos polished and on GitHub. Master portfolio README created. ROADMAP.md fully updated. Interview prep for 6 key questions done.

Day 42

Time	Activity	Detail
4.5h	Buffer	Interview prep continued: Deep dive on each projects technical decisions. Prepare for follow-up questions: Why not Qdrant? Why not CrewAI? What would you do at 10x scale? How would you add streaming? What if the LLM provider goes down? How do you handle German data residency? Write down your answers. Practice out loud. Time yourself (2 min per project).
4h	Buffer	Buffer: Revisit any from-scratch lessons that feel shaky. Re-code: self-attention (P7-L02), BPE tokenizer (P10-L01), RAG from scratch (P11-L06). These are the most likely whiteboard interview topics. If you can code these from memory, youre ready.

Deliverable: Interview answers prepared for follow-up questions. Core from-scratch implementations re-coded from memory.

Day 43

Time	Activity	Detail
4.5h	Buffer	Mock interview: Pick 3 projects. For each, spend 15 minutes explaining: architecture, tech choices, challenges, metrics, what youd improve. Record yourself. Watch the recording. Note: speaking pace, clarity, technical depth. Redo any that feel weak. This is the most important prep you can do.
4h	Buffer	Buffer: Final review of all from-scratch [CORE] lessons. Make a 1-page cheat sheet: transformer architecture, attention formula, BPE algorithm, RAG pipeline steps, agent loop, LoRA math, evaluation metrics, PII detection layers. This cheat sheet is for your own reference, not for memorization.

Deliverable: Mock interview recorded and reviewed. 1-page cheat sheet of core concepts created. Speaking pace and clarity assessed.

Day 44

Time	Activity	Detail
4.5h	Buffer	Final project review: Run docker compose up in R-P11 (capstone). Submit 5 test tickets. Verify the full pipeline works end-to-end. Screenshot the Streamlit dashboard for your portfolio. Take screenshots of 2-3 other project dashboards (R-P1 eval, R-P7 eval, R-P9 admin). Add these to your portfolio README.
4h	Buffer	Buffer: Rest day. Youve been at this for 44 days straight. Do something non-technical. Tomorrow is a final review day, not a work day. Your brain needs rest to consolidate 45 days of learning.

Deliverable: Capstone end-to-end verified. Portfolio screenshots taken. Rest day taken.

Day 45

Time	Activity	Detail
4.5h	Buffer	Final review day. Read through your CASE_STUDY.md. Read your 1-page cheat sheet. Skim each projects README. You should be able to explain any of the 12 projects in 2 minutes without looking at code. Review your ROADMAP.md - all 12 projects marked complete. You have: 12 portfolio projects, 8 Ed Donners projects, transformers/tokenizers/RAG/agents from scratch, a 15-20 page case study, 5 ADRs, a presentation deck outline.
4h	Buffer	Whats next: Start applying. Your portfolio is ready. Target roles: AI Engineer, LLM Engineer, RAG Engineer, ML Engineer (LLM focus), Data Engineer with AI focus. German market: emphasize GDPR compliance (R-P10), audit trails (R-P2, R-P11), enterprise patterns (R-P9 gateway, R-P8 prompt versioning). You are no longer an AI tourist - you are an AI engineer with 12 shipped projects.

Deliverable: 45-DAY PLAN COMPLETE. 12 roadmap projects shipped. Ed Donners course done. From-scratch fundamentals mastered. Case study written. Ready to apply for AI engineering roles.

Congratulations. You went from API caller to AI engineer in 45 days. Now go get the job.

8. 2-Month Extension (Days 46-60)

If you have 2 months instead of 1.5, use these 15 extra days for depth, re-dos, and interview prep.

Day	Focus	Activities
46-47	Re-do weak projects	Pick the 2 projects you understand least. Re-build them from scratch without AI assistance. This forces true understanding.
48-49	From-scratch [DEEP] lessons	Go through the [DEEP] lessons you skipped. P7-L08 (RoPE), P10-L05 (SentencePiece), P11-L09 (GRPO). These add depth that interviewers love.
50-51	Deployment	Deploy 2-3 projects to a real server (Hetzner Cloud, EUR4/month). Set up nginx reverse proxy, SSL with certbot, GitHub Actions CI/CD. A deployed project is 10x more impressive than a Docker Compose one.
52-53	Open-source contribution	Find an AI/LLM open-source project (LangChain, LlamaIndex, DSPy, Instructor). Fix a bug or add a feature. Even a small PR counts. This shows you can work with existing codebases.
54-55	Blog post	Write a technical blog post about one of your projects. Building a Production RAG System with pgvector and BM25 Hybrid Search. Post on Medium or your own blog. This is your public proof of expertise.
56-57	Mock interviews	Do 5 mock interviews. Use pramp.com or interview a friend. Focus on: system design (design a customer support AI system), ML depth (explain attention), coding (implement a semantic cache).
58-60	Job applications	Apply to 10-15 roles. Tailor each application to the JD. Link your portfolio. Reference specific projects that match the job requirements. German market: emphasize GDPR, audit trails, enterprise patterns.

9. AI Code Generation Workflow

The roadmap projects use AI to generate boilerplate. This is the disciplined workflow that ensures you actually understand what you ship. Follow this for every roadmap project.

Step 1: Read the Plan

Before generating any code, read the full IMPLEMENTATION_PLAN.md for the project. Understand: what are we building, what's the architecture, what are the database tables, what's the API surface. If you can't explain the project to someone after reading the plan, read it again.

Step 2: Generate in Small Chunks

Ask AI to generate one component at a time (e.g., write the DocumentLoader class). Never ask for the whole project at once. Small chunks = you can verify each piece. If a chunk is > 100 lines, ask for it in smaller pieces.

Step 3: Read Every Line

Before pasting generated code into your file, read every line. Ask yourself: What does this line do? Why is it here? If you can't answer, ask the AI to explain. If the explanation doesn't make sense, ask again or look it up. Never ship code you don't understand.

Step 4: Run + Test Immediately

After each component, run it. Write a quick test. Does it do what the plan says it should? If not, debug. Don't move to the next component until the current one works. This prevents compounding errors.

Step 5: Document Your Understanding

After finishing a component, write a 2-3 sentence comment at the top explaining what it does and why. This forces you to articulate your understanding. If you can't write the comment, you don't understand it yet.

Step 6: Connect Components

Once individual components work, connect them. This is where you'll find integration bugs. Debug them yourself - don't ask AI to fix integration bugs, because the bug is in how YOU connected things, and AI doesn't have that context.

Step 7: Write the README

The README is your proof of understanding. Write it yourself, not with AI. Include: architecture diagram, component descriptions, how to run, metrics. If you can't write the README, you don't understand the project.

*The golden rule: **AI generates code, you generate understanding.** If you ship 12 projects but can't explain any of them, you wasted 45 days. If you ship 12 projects and can explain every architectural decision, you're an AI engineer.*

10. What to Ruthlessly Skip

You already know fundamentals. These are the specific items to skip across all three resources.

Ed Donners - Skip These

- W1: Environment setup, Cursor/UV/Git walkthrough (you know your tools)
- W3: Google Colab + GPU setup (you know Colab)
- W6: Random Forest, XGBoost, Bag of Words, traditional ML baselines (you know these from data engineering)
- W6: MLOps general concepts (you know MLOps; focus on the LLM-specific parts)
- Any lecture where Ed spends > 5 minutes on basic Python or git

From-Scratch Curriculum - Skip These Phases Entirely

- Phase 0: Python Foundations (you know Python)
- Phase 1: Data Structures (you know data structures)
- Phase 2: Math for ML (you know linear algebra, calculus, probability)
- Phase 3: NumPy, Pandas, Matplotlib basics (you use these daily)

From-Scratch Curriculum - Skip These Lesson Types

- All [SKIP] lessons (marked for experienced developers)
- Most [DEEP] lessons in Phase 1 (unless youre shaky on a specific topic)
- P4 (CNNs): Lessons on convolution, pooling, ResNet - skip unless youll work with vision models
- P6 (RNNs): Lessons on vanilla RNN, LSTM, GRU - skip (transformers replaced RNNs)
- P8 (Diffusion): All lessons - skip unless youll work with image generation
- P9 (Reinforcement Learning): All lessons - skip (RLHF is covered conceptually in P10)
- P12 (Distributed Training): All lessons - skip (you wont train models at scale)
- P13 (Inference at Scale): Lessons on TensorRT, vLLM, model serving - skim only
- P15 (Security): Lessons on prompt injection - skim (R-P10 covers PII, which is more relevant)
- P16 (Future Directions): All lessons - skip (speculative, not actionable)

Roadmap Projects - Do Not Skip Any

All 12 roadmap projects are essential. They are the proof of your skills. Even if a project seems simple (e.g., P8 prompt versioning), it teaches patterns youll need for the capstone (P11).

Resources Reference

Ed Donners Course (Udemy)

8 weeks, ~33 hours video + 8 projects. Watch at 1.5x. Course name: LLM Engineering: Master AI, Large Language Models and Agents by Ed Donners.

AI Engineering from Scratch

17 phases, ~150 lessons. Study only [CORE] lessons from Phases 7, 10, 11, 14. Skip Phases 0-3 (fundamentals you know). Skim [DEEP] lessons only if relevant to current project.

AI-Engineering Roadmap

12 projects across 5 tiers. Each project has an IMPLEMENTATION_PLAN.md with: architecture, database schema, API design, implementation phases, tech stack. All projects use Docker Compose, PostgreSQL + pgvector, FastAPI, Streamlit.

Tech Stack Summary

Category	Technology	Where Used
LLM Provider	OpenAI (GPT-4o, GPT-4o-mini)	All projects
Alternative Provider	Anthropic Claude, Ollama (local)	R-P9 failover
Vector DB	PostgreSQL + pgvector	R-P1, P2, P3, P6, P11
Graph DB	Neo4j	R-P3 (GraphRAG)
Cache	Redis	R-P6, P9
Framework	LangGraph	R-P6 (multi-agent)
API	FastAPI	All projects
UI	Streamlit	All projects (dashboards)
Orchestration	Apache Airflow	R-P5 (document pipeline)
Embeddings	BGE-large-en-v1.5, CLIP	R-P1, P3
Re-ranker	BGE-reranker-large	R-P1
PII Detection	Presidio + spaCy	R-P10, P11
SQL Validation	sqlglot	R-P4
Prompt Templating	Jinja2	R-P8
Statistics	scipy + statsmodels	R-P8
Fine-tuning	TRL SFT Trainer, QLoRA	Ed Donners W7 (not roadmap)
Experiment Tracking	Weights & Biases	Ed Donners W7 (not roadmap)

Final Word

This plan is intense. 8.5 hours/day for 45 days is ~380 hours of focused study. Thats more than a university semester course. You will be tired. You will want to skip days. Dont. The roadmap projects are your portfolio - they are what gets you hired. The from-scratch lessons are your depth - they are what keeps you hired. Ed Donners is your breadth - he shows you whats possible. All three together make you an AI engineer.